

Manual Técnico del Sistema "Refugio Rodante"

(Basado en estándares IEEE 1063-2001 e ISO/IEC/IEEE 26511:2018)

Nombre del Proyecto: Refugio Rodante

Versión: 1.0

Fecha: [03/06/2025]

Autores:

Emanuel Burítica Guababe (Base de Datos)

Wilber de Jesús Pineda Valois (Desarrollador Backend)

Edna Patricia Castañeda (Desarrollador Frontend)

Miguel Angel Guzman (Pruebas)

Logo del SENA



Refugio Rodante



Tabla de Contenido

1. Introducción
2. Arquitectura del Sistema
3. Requisitos Técnicos
4. Estructura de la Base de Datos
5. Manual de Instalación
6. Pruebas Técnicas
7. Seguridad del Sistema
8. Mantenimiento
9. Control de Versiones
10. Anexos

Introducción

Propósito: Documentar la arquitectura, instalación y mantenimiento de la plataforma "Refugio Rodante" para reserva de estacionamientos.

Alcance: Sistema web basado en PHP, Laravel y MySQL, con integración de Google Maps API y Wompi.

Público Objetivo

- Técnicos: Desarrolladores y administradores de sistemas.
- Mantenedores: Equipos futuros que escalen el proyecto.

Convenciones.

- Código en negrita: Comandos o fragmentos clave
- Itálicas: Notas importantes

- Colores: Rojo para errores, verde para acciones exitosas.

Arquitectura del Sistema

Diagrama de Arquitectura (MVC).

Cliente (Frontend: Blade/Bootstrap) → Servidor (Laravel/PHP) → Base de Datos (MySQL)

↑ ↓

APIs Externas (Google Maps, Wompi)

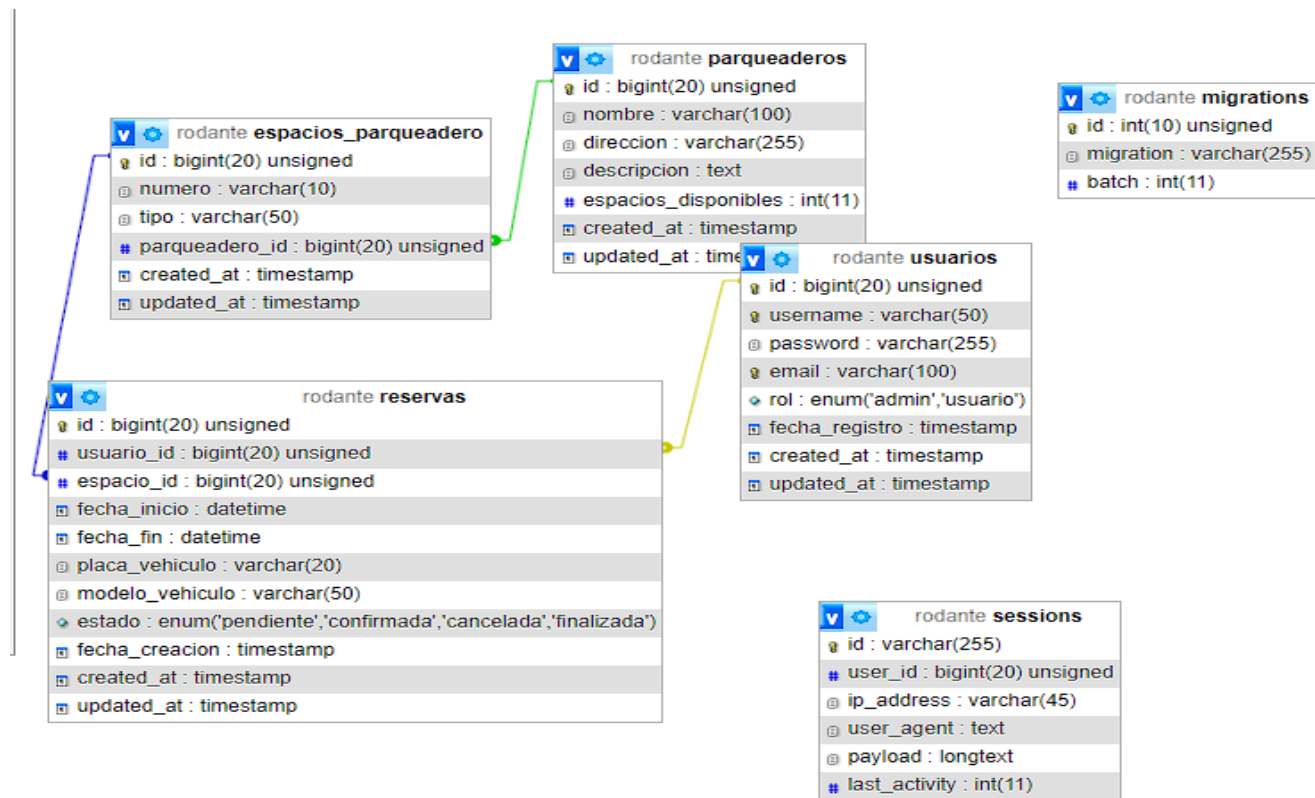
Tecnologías.

- **Backend:** Laravel 10.x, PHP 8.x
- **Frontend:** Blade, Bootstrap 5
- **Base de Datos:** MySQL (XAMPP)
- **APIs:** Google Maps JavaScript API, Wompi para pagos

Requisitos Técnicos hardware.

- **Hardware:** - Servidor: 2GB RAM, 20GB SSD. Cliente: Navegador moderno (Chrome/Firefox).
- **Software:** - XAMPP (Apache, MySQL, PHP), Composer, Git.
- **Dependencias:** - Laravel Breeze (autenticación), GuzzleHTTP (consumo de APIs).
- **Red:** - Conexión HTTPS, puertos 80 (HTTP) y 3306 (MySQL) abiertos.

Estructura de la Base de Datos diagrama entidad-relación (MER).



Tablas principales.

usuarios: id, username, email, contraseña, rol

parqueaderos: id, nombre, direccion, espacios_disponibles,

reservas: id, usuario_id, espacio_id, fecha_hora, estado,placa_vehiculo

Manual de Instalación requisitos

Clonar Repositorio:

```
git clone https://github.com/equipo-refugio-rodante/repo.git
```

```
cd repo
```

Instalar Dependencias:

```
composer install
```

```
npm install
```

Configurar .env:

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=rodante

DB_USERNAME=root

DB_PASSWORD=

GOOGLE_MAPS_API_KEY=tu_clave_aquí

WOMPI_API_KEY=tu_clave_aquí

Ejecutar Migraciones

```
php artisan migrate --  
seed
```

Pruebas Técnicas

- **Unitarias (PHPUnit):** Validación de lógica de reservas.
- **Funcionales (Postman):** Endpoints de la API REST.

Ejemplo de Prueba Unitaria:

```
public function testCrearReserva()
{
    $user = User::factory()->create();
    $response = $this->actingAs($user)->post('/reservas', [
        'parqueadero_id' => 1,
        'fecha_inicio' => now()->addDay()
    ]);
    $response->assertStatus(201);
}
```

Seguridad del Sistema

Autenticación: Laravel Breeze (sesiones cifradas).

Protecciones:

- **SQL Injection:** Uso de Eloquent ORM.
- **CSRF:** Middleware incluido en Laravel.
- **XSS:** Escape automático en Blade ({{ \$variable }}).
- **Cifrado:** Contraseñas con bcrypt, datos sensibles en .env.

Mantenimiento

Respaldos de BD:

```
mysqldump -u [usuario] -p [nombre_bd] > backup_$(date +%F).sql
```

Logs: Ubicación en storage/logs/laravel.log

Actualización:

```
git pull origin main
composer update
php artisan migrate
```

Control de Versiones

Repositorio: <https://github.com/SrAlucart/Versiones>

Ramas:

- **main:** Versión estable (solo releases).
- **dev:** Desarrollo activo (pull requests).

Anexos Técnicos

Endpoints API:

```
POST /api/reservas → Crear reserva
GET /api/parqueaderos → Listar parqueaderos disponibles
```

Fragmento de Código (Laravel):php

```
// En ParqueaderoController.php
public function index()
{
    return Parqueadero::select('id', 'nombre', 'latitud', 'longitud')->get();
}
```

Criterios de Calidad Cumplidos

Criterio	Cumplimiento
Claridad técnica	Lenguaje preciso, ejemplos prácticos y sin ambigüedades.
Estructura coherente	Secuencia lógica: desde requisitos hasta mantenimiento.
Documentación completa	Cubre BD, APIs, seguridad, pruebas y control de versiones.
Formato profesional	Numeración automática, tablas, diagramas y tipografía legible

 Repositorio GitHub: https://github.com/SrAlucart/Proyecto_y_Documentacion

Diagrama UML de Clases:

